

Category Theory

take home test

2026

I have written Q1-4, and formally verified them in Section 6. Below I have tried to rewrite these lean proofs into natural language proofs.

Contents

1. Question 1	2
2. Question 2	3
2.1. Functoriality Proofs	3
2.1.1. $H_l(X)$ is functorial	3
2.1.2. H_l is functorial	3
2.1.3. $H_r(X)$ is functorial	4
2.1.4. H_r is functorial	4
2.2. Naturality Proofs	4
2.2.1. $H_l(f)$ is natural	4
2.2.2. $H_r(f)$ is natural	4
2.3. $H_l \dashv H$	4
2.3.1. η_X naturality in X	5
2.3.2. $\varepsilon_{X,Y}$ naturality in X	5
2.3.3. $\varepsilon_{X,Y}$ naturality in Y	5
2.3.4. Left-triangle	5
2.3.5. Right-triangle	5
2.4. $H \dashv H_r$	6
3. Question 3	7
4. Question 4	8
4.1. $\text{iso} := \forall Pn, P(n)^n \times P(n+1) = \tilde{P}(n) \simeq \text{obj}(P^N) = (\lambda (n) \times N) \rightarrow P$	8
4.2. Defining the map $\tilde{P}(f) : \tilde{P}(n) \rightarrow \tilde{P}(m)$	9
4.3. $\tilde{P} \simeq P^N$	9
4.4. $\text{app} : \tilde{P} \times N \rightarrow P$	9
5. Question 5	10
6. Formally verified solutions	11
6.1. Q1	11
6.2. Q2	14
6.3. Q3	17
6.4. Q4	19

1. Question 1

[Link to a live lean version here](#)

The left adjoint of the functor $R^{(-)} : C^{\text{op}} \rightarrow C$, is the functor $R'^{(-)} : C \rightarrow C^{\text{op}}$, given by $R'^X \triangleq R^X$, and $R'^f \triangleq \text{cur} (\text{app} \circ (f \times_m \text{id}))$. As CCCs can get quite messy to work with I decided to leave it to the lean formalization.

2. Question 2

Link to a live lean version here

This task was solved much better in lean and I recommend looking at it there more.

We begin by constructing $H_l, H_r : \text{Set}^{\text{obj}(C)} \rightarrow \text{Set}^C$, which will be the adjoints of H .

$$\begin{aligned} H_l(X)(V) &\triangleq \sum_{U:C} C(U, V) \times XU \\ H_l(X)(f)(\langle U, m, x, \rangle) &\triangleq \langle U, f \circ m, x, \rangle \\ H_l(f)_V(\langle U, m, x, \rangle) &\triangleq \langle U, m, fx, \rangle \end{aligned}$$

$$\begin{aligned} H_r(X)(V) &\triangleq \prod_{U:C} C(V, U) \rightarrow XU \\ H_r(X)(f)(o)(U)(m) &\triangleq o \ U(m \circ f) \\ H_r(f)_V(o)(U)(m) &\triangleq f(oUm) \end{aligned}$$

The functoriality of H_l , H_r , $H_l(X)$ and $H_r(Y)$; and the naturality of $H_l(f)$ and $H_r(f)$ follow definitionally.

Functoriality Proofs.

These proofs were auto-solved in the Lean formalization, but I include them here anyway.

2.1.1. $H_l(X)$ is functorial

Identity:

$$\begin{aligned} H_l(X)(\text{id}_V)(\langle U, m, x, \rangle) &= \langle U, \text{id}_V \circ m, x, \rangle \\ &= \langle U, m, x, \rangle \end{aligned}$$

Composition:

$$\begin{aligned} H_l(X)(g \circ f)(\langle U, m, x, \rangle) &= \langle U, (g \circ f) \circ m, x, \rangle \\ &= \langle U, g \circ (f \circ m), x, \rangle \\ &= H_l(X)(g)(\langle U, f \circ m, x, \rangle) \\ &= H_l(X)(g)(H_l(X)(f)(\langle U, m, x, \rangle)) \end{aligned}$$

2.1.2. H_l is functorial

Identity:

$$\begin{aligned} (H_l(\text{id}_X))(\langle U, m, x, \rangle) &= \langle U, m, \text{id}_X(x), \rangle \\ &= \langle U, m, x, \rangle \end{aligned}$$

Composition:

$$\begin{aligned} (H_l(g \circ f))(\langle U, m, x, \rangle) &= \langle U, m, (g \circ f)(x), \rangle \\ &= \langle U, m, g(f(x)), \rangle \\ &= (H_l(g))(\langle U, m, f(x), \rangle) \\ &= (H_l(g))((H_l(f))(\langle U, m, x, \rangle)) \end{aligned}$$

2.1.3. $H_r(X)$ is functorial

Identity:

$$\begin{aligned} H_r(X)(\text{id}_V)(o)(U)(m) &= o \ U(m \circ \text{id}_V) \\ &= o \ U(m) \end{aligned}$$

Composition:

$$\begin{aligned} H_r(X)(g \circ f)(o)(U)(m) &= o \ U(m \circ (g \circ f)) \\ &= o \ U((m \circ g) \circ f) \\ &= H_r(X)(f)(o)(U)(m \circ g) \\ &= H_r(X)(f)(H_r(X)(g)(o))(U)(m) \end{aligned}$$

2.1.4. H_r is functorial

Identity:

$$\begin{aligned} (H_r(\text{id}_X))(o)(U)(m) &= \text{id}_X(o \ U m) \\ &= o \ U m \end{aligned}$$

Composition:

$$\begin{aligned} (H_r(g \circ f))(o)(U)(m) &= (g \circ f)(o \ U m) \\ &= g(f(o \ U m)) \\ &= (H_r(g))((H_r(f))(o))(U)(m) \end{aligned}$$

Naturality Proofs.

Likewise here

2.2.1. $H_l(f)$ is natural

RTP: $H_l(Y)(g) \circ H_l(f)_V = H_l(f)_W \circ H_l(X)(g)$

$$\begin{aligned} (H_l(Y)(g) \circ H_l(f)_V)(\langle U, m, x \rangle) &= H_l(Y)(g)(\langle U, m, fx \rangle) \\ &= \langle U, g \circ m, fx \rangle \\ &= H_l(f)_W(\langle U, g \circ m, x \rangle) \\ &= H_l(f)_W(H_l(X)(g)(\langle U, m, x \rangle)) \\ &= (H_l(f)_W \circ H_l(X)(g))(\langle U, m, x \rangle) \end{aligned}$$

2.2.2. $H_r(f)$ is natural

RTP: $H_r(Y)(g) \circ H_r(f)_V = H_r(f)_W \circ H_r(X)(g)$

$$\begin{aligned} (H_r(Y)(g) \circ H_r(f)_V)(o)(U)(m) &= H_r(Y)(g)(H_r(f)_V(o))(U)(m) \\ &= H_r(f)_V(o)(U)(m \circ g) \\ &= f(o \ U(m \circ g)) \\ &= H_r(f)_W(o \ U(m \circ g)) \\ &= H_r(f)_W(H_r(X)(g)(o)(U)(m)) \\ &= (H_r(f)_W \circ H_r(X)(g))(o)(U)(m) \end{aligned}$$

$H_l \dashv H$.

We start by defining the unit $\eta : \text{id} \rightarrow HH_l$ and the counit $\varepsilon : H_l H \rightarrow \text{id}$, We pick $\eta_X(V)(v) \triangleq \langle V, \text{id}, v \rangle$ and $\varepsilon_{X,Y}(\langle V, f, o \rangle) \triangleq X(f)o$.

2.3.1. η_X naturality in X

RTP: $H(H_l(f)) \circ \eta_X = \eta_Y \circ f$

$$\begin{aligned} (H(H_l(f)) \circ \eta_X)(V)(v) &= H(H_l(f))(\eta_X(V)(v)) \\ &= H(H_l(f))(\langle V, \text{id}, v \rangle) \\ &= H_l(f)_V(\langle V, \text{id}, v \rangle) \\ &= \langle V, \text{id}, fVv \rangle \\ &= \eta_Y(V)(fVv) \\ &= (\eta_Y \circ f)(V)(v) \end{aligned}$$

2.3.2. $\varepsilon_{X,Y}$ naturality in X

RTP: $f_V \circ \varepsilon_{X,V} = \varepsilon_{Y,V} \circ H_l(f)_V$ for $f : X \rightarrow Y$ natural transformation

$$\begin{aligned} (f_V \circ \varepsilon_{X,V})(\langle U, m, x \rangle) &= f_V(X(m)(x)) \\ &= (Y(m) \circ f_U)(x) && \text{by naturality of } f \\ &= Y(m)(f_U(x)) \\ &= \varepsilon_{Y,V}(\langle U, m, f_U x \rangle) \\ &= (\varepsilon_{Y,V} \circ H_l(f)_V)(\langle U, m, x \rangle) \end{aligned}$$

2.3.3. $\varepsilon_{X,Y}$ naturality in Y

RTP: $X(f) \circ \varepsilon_{X,U} = \varepsilon_{X,V} \circ H_l(X)(f)$ for $f : U \rightarrow V$

$$\begin{aligned} (X(f) \circ \varepsilon_{X,U})(\langle W, m, x \rangle) &= X(f)(X(m)(x)) \\ &= X(m \circ f)(x) && \text{by functoriality of } X \\ &= \varepsilon_{X,V}(\langle W, m \circ f, x \rangle) \\ &= (\varepsilon_{X,V} \circ H_l(X)(f))(\langle W, m, x \rangle) \end{aligned}$$

2.3.4. Left-triangle

RTP: $\varepsilon_{H_l(X),V} \circ (H_l(\eta_X))_V = \text{id}_{H_l(X)(V)}$

$$\begin{aligned} (\varepsilon_{H_l(X),V} \circ (H_l(\eta_X))_V)(\langle U, m, x \rangle) &= \varepsilon_{H_l(X),V}(\langle U, m, \eta_X(U)(x) \rangle) \\ &= \varepsilon_{H_l(X),V}(\langle U, m, \langle U, \text{id}, x \rangle \rangle) \\ &= H_l(X)(m)(\langle U, \text{id}, x \rangle) \\ &= \langle U, m \circ \text{id}, x \rangle \\ &= \langle U, m, x \rangle \end{aligned}$$

2.3.5. Right-triangle

RTP: $(H(\varepsilon_X))_U \circ \eta_{H(X),U} = \text{id}_{H(X)(U)}$

$$\begin{aligned} ((H(\varepsilon_X))_U \circ \eta_{H(X),U})(v) &= H(\varepsilon_X)(\eta_{H(X),U}(v)) \\ &= \varepsilon_{X,U}(\langle U, \text{id}, v \rangle) \\ &= X(\text{id})(v) \\ &= v \end{aligned}$$

$H \dashv H_r$.

We start by defining the unit $\eta : \text{id} \rightarrow H_r H$ and the counit $\varepsilon : H H_r \rightarrow \text{id}$, We pick $\eta_{X,Y}(v)(U)(m) \triangleq X(m)v$ and $\varepsilon_X(V)(h) \triangleq hV \text{id}$.

Next We need to prove naturalities and the two triangle equalities.

The proofs for naturality and triangles are equally long and They are fully formalised in the lean proof.

3. Question 3

Link to a live lean version here

To begin let us consider the F -algebra transformer $\tilde{F} : F_{\text{Alg}} \rightarrow F_{\text{Alg}} \triangleq \lambda \langle A, \alpha \rangle. \langle F(A), F(\alpha) \rangle$ on this we define the morphism $\text{down} : \tilde{F}I \rightarrow I \triangleq \iota$, which by definition satisfies the required equation.

Now we can proceed to show that ι is an isomorphism.

Proof:

- **RTP:** $\exists m : C(I, FI), m \circ \iota = \text{id}_{FI} \wedge \iota \circ m = \text{id}_I$
- **Use:** $(\llbracket \tilde{F}I \rrbracket_h) : C(I, FI)$
- **Case :** $\iota \circ (\llbracket \tilde{F}I \rrbracket_h) = \text{id}_{FI}$
 - $\iota \circ (\llbracket \tilde{F}I \rrbracket_h) = (\text{down} \circ \llbracket \tilde{F}I \rrbracket_h)$ by definition
 - $= (\llbracket I \rrbracket_h)$ by composition of initial morphism
 - $= (\text{id}_I)_h$ by uniqueness of initial morphism
 - $= \text{id}_I$ by definition
- **Case :** $(\llbracket \tilde{F}I \rrbracket_h) \circ \iota = \text{id}_I$
 - $(\llbracket \tilde{F}I \rrbracket_h) \circ \iota = F(\iota) \circ F(\llbracket \tilde{F}I \rrbracket_h)$ by the homomorphism requirement on ι
 - $= F(\iota \circ (\llbracket \tilde{F}I \rrbracket_h))$ by F functorality
 - $= F(\text{id}_{FI})$ by the previous case
 - $= \text{id}_I$ by F functorality

4. Question 4

Link to a live lean version here

To begin, let us define the function

$$\text{extend } (f : [n] \hookrightarrow [m])(p : [m])(h : p \notin f^{-1}) : [n+1] \hookrightarrow [m] = \begin{cases} p & \text{if } 0 \\ f n & \text{if } n+1 \end{cases}$$

this is injective by definition using the argument h .

Now we will begin by showing the object isomorphism (note I use $\mathcal{K}(n)$ to represent the coyoneda functor at n).

$$\text{iso} := \forall P n, P(n)^n \times P(n+1) = \tilde{P}(n) \simeq \text{obj}(P^N) = (\mathcal{K}(n) \times N) \rightarrow P.$$

Let us first define $u : P(n)^n \times P(n+1) \rightarrow \text{Set}^C((\mathcal{K}(n) \times N), P)$ as

$$u(\langle o_l, o_r \rangle)_m(\langle p_l, p_r \rangle) : P(m) \triangleq \begin{cases} P(p_l)(o_l v) & \text{if } \exists v, p_l v = p_r \\ P(\text{extend } p_l p_r _) o_r & \text{otherwise} \end{cases}$$

We need to show the naturality of $u(\langle o_l, o_r \rangle)_m$, for this consider a function $f : m \rightarrow m'$.

- **RTP:** $u(\langle o_l, o_r \rangle)_m \circ \mathcal{K}(f)_{n \times f} = P(f) \circ u(\langle o_l, o_r \rangle)_{m'}$
- **Function extentionality** with $(\langle p_l, p_r \rangle)$
- **Case** with $\exists v, f(p_l v) = f p_r$,
 - **RTP:** $P(f \circ p_l)(o_r v) = P(f)(P(p_l)(o_r v))$
 - Which is true by functorality in P
- **Case** with $\neg \exists v, f(p_l v) = f p_r$,
 - **RTP:** $P(\text{extend } (f \circ p_l)(f p_r) _) o_r = (P(f) \circ P(\text{extend } p_l p_r _)) o_r = P(f \circ \text{extend } p_l p_r _) o_r$
 - **By congruence**
 - **RTP:** $(\text{extend } (f \circ p_l)(f p_r) _) = f \circ \text{extend } p_l p_r _$
 - **Function extentionality** with i
 - **Case** $i = 0$
 - **RTP:** $f p_r = f p_r$
 - True by refl
 - **Case** $i = n+1$
 - **RTP:** $f(p_l n) = f(p_l n)$
 - True by refl

We then define $v : \text{Set}^C((\mathcal{K}(n) \times N), P) \rightarrow P(n)^n \times P(n+1)$ as

$$v(t) : P(n)^n \times P(n+1) \triangleq \langle \lambda i. t_n \langle \text{id}, i \rangle, t_{n+1} \langle \text{succ}, 0 \rangle \rangle$$

Now we have to prove that $u \circ v = \text{id}$

- **RTP:** $(u(vt))_m(\langle i_l, i_r \rangle) = \begin{cases} P(i_l)(t_n(\langle \text{id}, v \rangle)) & \text{if } \exists v, i_l v = i_r \\ P(\text{extend } i_l i_r _)(t_{n+1}(\langle \text{succ}, 0 \rangle)) & \text{otherwise} \end{cases} = t_m(\langle i_l, i_r \rangle)$
- **Case** $\exists v, i_l v = i_r$
 - $(P(i_l) \circ t_n)(\langle \text{id}, v \rangle) = t_m(\langle i_l \circ \text{id}, i_l v \rangle)$ by naturality of t
 - $= t_m(\langle i_l, i_r \rangle)$ by defn of v
 - $= t_m(\langle i_l, i_r \rangle)$
- **Case** otherwise
 - $(P(\text{extend } i_l i_r _) \circ t_{n+1})(\text{succ}, 0) = t_m(\text{extend } i_l i_r _ \circ \text{succ}, \text{extend } i_l i_r _ 0)$ by naturality of t
 - $= t_m(\text{extend } i_l i_r _ \circ \text{succ}, i_r)$ by defn of extend
 - $= t_m(i_l, i_r)$ by defn of extend

Finally $v \circ u = \text{id}$,

- **RTP**: $v(u(\langle i_l, i_r \rangle)) = \langle i_l, i_r \rangle$
- using **product extentionality**
- **Case**: **fst RTP**: $(v(u(\langle i_l, i_r \rangle))).1 = i_l$
 - apply **Function extentionality** with j
 - **RTP**: $\begin{cases} P(\text{id})(i_l j) & \text{if } \exists v, \text{id } v = i \\ P(\text{extend } (\text{id}) i _) i_r & \end{cases} = i_l j$
 - There exists a $\text{id } v = i$, exactly i
 - **RTP**: $P(\text{id})(i_l j) = i_l j$
 - $P(\text{id})(i_l j) = \text{id}(i_l j) = i_l j$
- **Case**: **snd RTP**: $(v(u(\langle i_l, i_r \rangle))).2 = i_r$
 - **RTP**: $\begin{cases} P(\text{succ})(i_l v) & \text{if } \exists v, \text{succ } v = 0 \\ P(\text{extend succ } 0 _) i_r & \text{otherwise} \end{cases} = i_r$
 - There doesn't exist some v such that $v + 1 = 0$
 - **RTP**: $P(\text{extend succ } 0 _) i_r = i_r = \text{id } i_r = P(\text{id}) i_r$
 - By **congruence** we are
 - **RTP**: $\text{extend succ } 0 _ = \text{id}$
 - apply **Function extentionality** with i
 - **Case** $i = 0$
 - $0 = \text{id } 0$
 - **Case** $i = n + 1$
 - $n + 1 = \text{id}(n + 1)$

We are now done and have the isomorphism iso .

Defining the map $\tilde{P}(f) : \tilde{P}(n) \rightarrow \tilde{P}(m)$.

We define the map as $P(f) \triangleq \text{iso}_{\text{symm}} \circ P^N(f) \circ \text{iso}$, its functorality is easy to prove

$$\begin{aligned}
 \tilde{P}(\text{id}) &= \text{iso}_{\text{symm}} \circ P^N(\text{id}) \circ \text{iso} \\
 &= \text{iso}_{\text{symm}} \circ \text{id} \circ \text{iso} && \text{By functorality of } P^N \\
 &= \text{iso}_{\text{symm}} \circ \text{iso} \\
 &= \text{id}
 \end{aligned}$$

$$\begin{aligned}
 \tilde{P}(f \circ g) &= \text{iso}_{\text{symm}} \circ P^N(f \circ g) \circ \text{iso} \\
 &= \text{iso}_{\text{symm}} \circ P^N(f) \circ P^N(g) \circ \text{iso} && \text{By functorality of } P^N \\
 &= \text{iso}_{\text{symm}} \circ P^N(f) \circ \text{id} \circ P^N(g) \circ \text{iso} \\
 &= \text{iso}_{\text{symm}} \circ P^N(f) \circ \text{iso} \circ \text{iso}_{\text{symm}} \circ P^N(g) \circ \text{iso} \\
 &= \tilde{P}(f) \circ \tilde{P}(g)
 \end{aligned}$$

$$\tilde{P} \simeq P^N.$$

To construct the isomorphism, we simply have to prove that iso is natural. By the way we picked the morphism map we get both of these for free: $\text{iso} \circ \tilde{P}(f) = \text{iso} \circ \text{iso}_{\text{symm}} \circ P^N(f) \circ \text{iso} = P^N(f) \circ \text{iso}$, and the other direction follows by analogy.

$$\text{app} : \tilde{P} \times N \rightarrow P.$$

We define app quite simply as $\text{app} \triangleq \text{app}_{P^N} \circ (\text{iso} \times \text{id})$. By construction, this satisfies the required universal property.

5. Question 5

I have identified that the structure $S(I)$ is the Schanuel topos (nominal sets). Therefore the functor we are looking for is the sheafification functor. I have desperately tried to construct this. I see the object of the functor will be some quotient over what looks like products of a hom and an object of the presheaf, the exact object and relation are lost to me. I sadly think my knowledge of the course is not sufficient for this.

6. Formally verified solutions

Q1.

```
1  import Mathlib.CategoryTheory.Category.Cat.CartesianClosed
2  import Mathlib.CategoryTheory.Monoidal.Braided.Basic
3
4  open CategoryTheory
5
6  universe u
7
8  variable {C : Type u} [Category C]
9      [MonoidalCategory C] [MonoidalClosed C]
10      (R : C)
11
12  open scoped MonoidalCategory
13  open scoped MonoidalClosed
14
15  section
16
17  variable {X Y Z W : C}
18
19  open MonoidalClosed
20
21  def app : X  $\otimes$  (X  $\rightarrow$ [_] Y)  $\rightarrow$  Y := uncurry <|  $\mathbb{1}$  _
22
23  theorem app_ev : (app : X  $\otimes$  _  $\rightarrow$  Y) = (ihom.ev _).app _ := by
24      rw [app, MonoidalClosed.uncurry_id_eq_ev]
25
26  @[simp]
27  theorem app_curry : curry app =  $\mathbb{1}$  (X  $\rightarrow$ [_] Y) := by
28      rw [app, curry_eq_iff]
29
30  @[simp]
31  theorem curry_appl (f : X  $\otimes$  Y  $\rightarrow$  Z) : X  $\triangleleft$  curry f  $\gg$  app = f := by
32      rw [app_ev]
33      rw [←uncurry_eq]
34      rw [←@eq_curry_iff]
35
36  @[simp]
37  theorem curry_appr {g : W  $\rightarrow$  Y} (f : Y  $\otimes$  X  $\rightarrow$  Z) : (g  $\otimes_m$  curry f)  $\gg$  app = (g  $\triangleright$  _)  
     $\gg$  f := by
38      rw [MonoidalCategory.tensorHom_def_assoc]
39      rw [app_ev]
40      rw [← @uncurry_eq]
41      rw [←@uncurry_curry]
42
```

```

43 end
44
45 variable [SymmetricCategory C]
46 open BraidedCategory MonoidalCategory
47
48 def pre : Cop ⇒ C where
49   obj v := (ihom v.unop).obj R
50   map f := (MonoidalClosed.pre f.unop).app R
51
52 open MonoidalClosed in
53 def adj : C ⇒ Cop where
54   obj X := .op <| X →[_] R
55   map f := .op <| curry <| (f ▷ _) » app
56   map_id X := by
57     rw [id_whiskerRight, Category.id_comp, app_curry, op_id]
58   map_comp {X Y Z} f g := by
59     apply (Opposite.op.injEq _ _).mpr
60     apply (curry_eq_iff _ _).mpr
61   calc
62     (f » g) ▷ (Z →[_] R) » app
63     = f ▷ (Z →[_] R) » g ▷ (Z →[_] R) » app
64                                     := by rw
65                                     [comp_whiskerRight_assoc]
66   _ = (f ⊗m curry (g ▷ (Z →[_] R) » app)) » app
67                                     := by rw [curry_app]
68   _ = X <| curry (g ▷ (Z →[_] R) » app) » f ▷ (Y →[_] R) » app
69                                     := by rw [tensorHom_def'_assoc]
69   _ = X <| curry (g ▷ (Z →[_] R) » app) » uncurry (curry (f ▷ (Y →[_] R) »
    app))
70                                     := by rw [uncurry_curry]
71   _ = uncurry (curry (g ▷ (Z →[_] R) » app) » curry (f ▷ (Y →[_] R) »
    app))
72                                     := by rw [uncurry_natural_left]
73
74 example : adj R ⊣ pre R := by
75   exact .mkOfUnitCounit {
76     unit := {
77       app X := show X → (ihom ((ihom X).obj R)).obj R
78               from MonoidalClosed.curry ((β_ _ _).hom » app)
79       naturality X Y f := by
80         dsimp [pre, adj]
81         rw [MonoidalClosed.curry_pre_app]
82         rw [braiding_naturality_left_assoc, curry_appl]
83         rw [←braiding_naturality_right_assoc]
84         rw [MonoidalClosed.curry_natural_left]
85     }

```

```

86     counit := {
87       app := fun (X) => .op
88       <| show X → (X →[_] R) →[_] R from MonoidalClosed.curry ((β_ _).hom >>
          app)
89       naturality X Y f := by
90         apply (Opposite.op.injEq _ _).mpr
91         dsimp [adj, pre]
92         rw [← MonoidalClosed.curry_pre_app]
93         rw [@app_curry, Category.id_comp]
94         rw [MonoidalClosed.curry_pre_app]
95         rw [← MonoidalClosed.curry_natural_left]
96         rw [braiding_naturality_left_assoc]
97         rw [braiding_naturality_right_assoc]
98         apply (MonoidalClosed.curry_eq_iff _ _).mpr
99         rw [MonoidalClosed.uncurry_curry, Iso.cancel_iso_hom_left]
100        rw [app_ev, app_ev]
101        rw [MonoidalClosed.id_tensor_pre_app_comp_ev]
102    }
103    left_triangle := by
104      ext
105      apply (Opposite.op.injEq _ _).mpr
106      dsimp [adj, pre]
107      rw [← MonoidalClosed.curry_natural_left]
108      apply (MonoidalClosed.curry_eq_iff _ _).mpr
109      change _ = app
110      rw [Category.comp_id]
111      rw [← tensorHom_def'_assoc]
112      rw [curry_appr]
113      rw [braiding_naturality_left_assoc, curry_appl]
114      rw [SymmetricCategory.symmetry_assoc]
115    right_triangle := by
116      ext (X)
117      dsimp [pre, adj]
118      rw [Category.id_comp]
119      rw [MonoidalClosed.curry_pre_app]
120      rw [braiding_naturality_left_assoc, curry_appl]
121      rw [SymmetricCategory.symmetry_assoc, app_curry]
122  }
123

```

Q2.

```
1  import Mathlib.CategoryTheory.Yoneda
2  import Mathlib.CategoryTheory.Category.Cat.CartesianClosed
3
4  open CategoryTheory
5
6  universe u
7
8  variable {C : Type u} [SmallCategory C]
9
10 def H : (C  $\Rightarrow$  Type u)  $\Rightarrow$  (C  $\rightarrow$  Type u) where
11   obj := Functor.obj
12   map := NatTrans.app
13
14 def HL : (C  $\rightarrow$  Type u)  $\Rightarrow$  (C  $\Rightarrow$  Type u) where
15   obj X := {
16     obj V :=  $\Sigma$  U, (U  $\rightarrow$  V)  $\times$  X U
17     map f := fun <U, m, x> => <U, m  $\gg$  f, x>
18   }
19   map {X Y} f := {
20     app V := fun <U, m, x> => <U, m, f _ x>
21   }
22
23 def HR : (C  $\rightarrow$  Type u)  $\Rightarrow$  (C  $\Rightarrow$  Type u) where
24   obj X := {
25     obj V := (U : C)  $\rightarrow$  (V  $\rightarrow$  U)  $\rightarrow$  X U
26     map f o U m := o U (f  $\gg$  m)
27   }
28   map {X Y} f := {
29     app V o U m := f _ (o U m)
30   }
31
32 instance : HL  $\dashv$  H (C := C) := .mkOfUnitCounit {
33   unit := {
34     app X V v := <V, 1 _, v>
35   }
36   counit := {
37     app X := {
38       app Y v := X.map v.2.1 v.2.2
39       naturality U V f := by
40         ext v
41         calc
42           X.map (v.2.1  $\gg$  f) v.2.2
43             = (X.map v.snd.1  $\gg$  X.map f) v.snd.2 := by rw [Functor.map_comp]
44             _ = X.map f (X.map v.snd.1 v.snd.2) := rfl
```

```

45     }
46     naturality U V f := by
47       ext X h
48       calc
49         (f.app h.fst >> V.map h.snd.1) h.snd.2
50         = (U.map h.snd.1 >> f.app X) h.snd.2 := by rw [f.naturality]
51   }
52   left_triangle := by
53     ext U V v
54     change (U_1 : C) × (U_1 → V) × U U_1 at v
55     calc
56       ((v.fst, (1 v.fst >> v.snd.1, v.snd.2)) : (U_1 : C) × (U_1 → V) × U U_1)
57       = (v.fst, (v.snd.1, v.snd.2)) := by rw [Category.id_comp]
58     _ = v := rfl
59   right_triangle := by
60     ext V U v
61     change V.obj U at v
62     calc
63       V.map (1 U) v
64       = 1 (V.obj U) v := by rw [V.map_id]
65     _ = v := rfl
66   }
67
68 instance : H (C := C) → HR :=
69   .mkOfUnitCounit {
70     unit := {
71       app X := {
72         app Y v U m := X.map m v
73         naturality U V f := by
74           ext o
75           funext U v
76           change (X.map f >> X.map v) o = X.map (f >> v) o
77           rw [X.map_comp]
78       }
79       naturality U V f := by
80         ext X x
81         funext Y m
82         change (f.app X >> V.map m) x = (U.map m >> f.app Y) x
83         rw [f.naturality]
84     }
85     counit := {
86       app X V h := h V (1 _)
87     }
88     left_triangle := by
89       ext V U v
90       change V.map (1 U) v = (1 (V.obj U)) v

```

```

91     rw [V.map_id]
92     right_triangle := by
93     ext U V v
94     funext X m
95     change v X (m >> 1 X) = v X m
96     rw [Category.comp_id]
97   }
98

```


Q3.

```

1  import Mathlib.CategoryTheory.Limits.Shapes.Terminal
2
3  universe u
4
5  open CategoryTheory
6
7  variable {C : Type u} [Category C] (F : C  $\Rightarrow$  C)
8
9  structure Alg where
10   A : C
11    $\alpha$  : F.obj A  $\rightarrow$  A
12
13  structure AlgHom (A B : Alg F) where
14   h : A.A  $\rightarrow$  B.A
15   eq : F.map h  $\gg$  B. $\alpha$  = A. $\alpha$   $\gg$  h
16
17  def AlgHom.comp
18   {X Y Z : Alg F}
19   (f : AlgHom F X Y)
20   (g : AlgHom F Y Z)
21   : AlgHom F X Z where
22   h := f.h  $\gg$  g.h
23   eq := calc
24     F.map (f.h  $\gg$  g.h)  $\gg$  Z. $\alpha$ 
25     = F.map f.h  $\gg$  F.map g.h  $\gg$  Z. $\alpha$  := by simp
26     _ = (F.map f.h  $\gg$  Y. $\alpha$ )  $\gg$  g.h := by rw [g.eq, Category.assoc]
27     _ = X. $\alpha$   $\gg$  f.h  $\gg$  g.h := by rw [f.eq, Category.assoc]
28
29  instance : Category (Alg F) where
30   Hom := AlgHom F
31   id _ := { h :=  $\mathbb{1}$  _, eq := by simp}
32   comp {X Y Z} f g := AlgHom.comp F f g
33
34   comp_id f := by simp [AlgHom.comp]
35   id_comp f := by simp [AlgHom.comp]
36   assoc f := by simp [AlgHom.comp]
37
38  def FUp (v : Alg F) : Alg F where
39   A := F.obj v.A
40    $\alpha$  := F.map v. $\alpha$ 
41
42  @[simp]
43  theorem FUpA {0} : (FUp F 0).A = F.obj 0.A :=
44   rfl

```

```

45
46 @[simp]
47 theorem FUpα {0} : (FUp F 0).α = F.map 0.α :=
48   rfl
49
50 open Limits
51
52 variable [HasInitial (Alg F)]
53
54 noncomputable def down : FUp F (⊥_ Alg F) → ⊥_ Alg F where
55   h := (⊥_ Alg F).α
56   eq := rfl
57
58 example [HasInitial (Alg F)] : IsIso ((⊥_ (Alg F)).α) where
59   out := (
60     (initial.to (FUp F (⊥_ _))).h,
61     by
62       have rhs : ((initial.to (FUp F (⊥_ Alg F))).h >> (⊥_ Alg F).α) = 11 _ :=
63         calc
64           (initial.to (FUp F (⊥_ Alg F)) >> down F).h
65           = (initial.to (⊥_ Alg F)).h := by rw [initial.to_comp (down F)]
66           _ = AlgHom.h (11 (⊥_ Alg F)) := by rw
67             [initial.hom_ext (11 (⊥_ Alg F))
68              (initial.to (⊥_ Alg F))]
69           constructor
70           · calc
71             (⊥_ Alg F).α >> (initial.to (FUp F (⊥_ Alg F))).h
72             = F.map (initial.to (FUp F _)).h >> (FUp F (⊥_ _)).α := by rw
73               [F.map_comp]
74             _ = F.map (11 _) := by rw
75               [rhs]
76             _ = 11 _ := by rw
77               [Functor.map_id]
78             · exact rhs,
79   )

```

Q4.

```
1  import Mathlib.CategoryTheory.Monoidal.Closed.Types
2
3  open CategoryTheory
4
5  open scoped MonoidalCategory
6
7  section
8
9  universe u
10 variable {C : Type} [SmallCategory C]
11
12 def adjv (V : C ⇒ Type) : ((C ⇒ Type) × C) ⇒ Type where
13   obj X := (coyoneda.obj (.op X.2) ⊗ V) → X.1
14   map {X Y} f n := ((coyoneda.map (.op f.2)) ⊗m (11 _)) >> n >> f.1
15
16 axiom cant_be_bothered {V : C ⇒ Type} : MonoidalCategory.tensorLeft V ⊣
  Functor.curry.obj (adjv V)
17
18 noncomputable instance : MonoidalClosed (C ⇒ Type) where
19   closed V := {
20     rightAdj := Functor.curry.obj <| adjv V
21     adj := cant_be_bothered
22   }
23
24 end
25
26 open Function (Embedding)
27
28 instance (priority := 10000) : Category Nat where
29   Hom n m := Fin n ↔ Fin m
30   id n := Embedding.refl _
31   comp := Embedding.trans
32
33 variable (P : Nat ⇒ Type)
34
35 def N : Nat ⇒ Type where
36   obj n := Fin n
37   map := Embedding.toFun
38
39 section
40
41 noncomputable def inPre {m n : Nat} (f : Fin m ↔ Fin n) (i : Fin n) : Option
  (Fin m) :=
42   (Finset.univ.filter (f · = i)).val.toList.head?
43
```

```

44 theorem inPre_filter_eq {m n : Nat} (f : Fin m → Fin n) (i : Fin n)
45   : Finset.univ.filter (f · = i) = if h : ∃ v, f v = i then {Classical.choose
46     h} else {} := by
47   split <|> rename_i heq
48   · ext n
49     simp only [Finset.mem_filter, Finset.mem_univ, true_and,
50       Finset.mem_singleton]
51     constructor <|> rintro rfl
52     · exact (f.injective <| Classical.choose_spec heq).symm
53     · exact Classical.choose_spec heq
54   · ext n
55     simp only [not_exists, Finset.mem_filter, Finset.mem_univ, true_and,
56       Finset.notMem_empty,
57       iff_false] at heq ⊢
58     apply heq
59
60 theorem inPre_eq {m n : Nat} (f : Fin m → Fin n) (i : Fin n) : inPre f i
61   = if h : ∃ v, f v = i then Option.some (Classical.choose h) else .none :=
62   by
63   dsimp only [inPre]
64   rw [inPre_filter_eq]
65   split <|> simp
66
67 @[simp]
68 theorem inPre_refl {n : Nat} {i} : inPre (1 n) i = .some i := by
69   simp [inPre, CategoryStruct.id, Finset.filter_eq']
70
71 @[simp]
72 theorem inPre_succEmb {n : Nat} : inPre (Fin.succEmb n) 0 = none := by
73   simp [inPre]
74
75 @[simp]
76 theorem inPre_iff {n m} {f : Fin n → Fin m} {i v}
77   : (inPre f i = some v) ↔ f v = i where
78   mp h := by
79     simp only [inPre_eq, Option.dite_none_right_eq_some, Option.some.injEq] at
80       h
81     rcases h with ⟨h, rfl⟩
82     exact Classical.choose_spec h
83   mpr := by
84     rintro rfl
85     simp [inPre, Finset.filter_eq']
86
87 @[simp]
88 theorem inPre_f {n m} {f : Fin n → Fin m} {i}
89   : inPre f (f i) = .some i := by simp
90

```

```

86 @[simp]
87 theorem ninPre_iff {n m} {f : Fin n → Fin m} {i}
88   : (inPre f i = none) ↔ ∀ x, f x ≠ i where
89   mp h i := by rintro rfl; simp at h
90   mpr h := Option.eq_none_iff_forall_some_ne.mpr
91   fun a h' => h a (inPre_iff.mp h'.symm)
92
93 @[simp]
94 theorem inPre_defn {n m} {f : Fin n → Fin m} {i}
95   : inPre f (f i) = .some i := by
96   simp
97
98 @[simp]
99 theorem inPre_comp {n m k}
100   {f : Fin n → Fin m}
101   {g : Fin m → Fin k}
102   {v : Fin _}
103   : inPre (f.trans g) (g v) = inPre f v := by
104   simp [inPre_eq]
105
106 end
107
108 def PSq0 (n : Nat) : Type := (Fin n → P.obj n) × P.obj (n + 1)
109
110 noncomputable def extend {n m : Nat}
111   (map : n → m)
112   (v : Fin m)
113   (h : inPre map v = none)
114   : n + 1 → m where
115   toFun := Fin.cases v map.toFun
116   inj' {a b} h' := by
117     rw [ninPre_iff] at h
118     simp only [Embedding.toFun_eq_coe] at h'
119     induction a using Fin.cases
120     <=> induction b using Fin.cases
121     <=> simp only [Fin.cases_zero, Fin.cases_succ] at h'
122     · rfl
123     · exact (h _ h'.symm).elim
124     · exact (h _ h').elim
125     · rw [EmbeddingLike.apply_eq_iff_eq] at h'
126       subst h'
127       rfl
128
129 noncomputable def objIso {P} {n : Nat} : PSq0 P n ≈ (N →[_] P).obj n where
130   toFun pqsv := {
131     app m prod := by

```

```

132     change (n → m) ⊗ Fin m at prod
133     change P.obj m
134     exact match h : inPre prod.1 prod.2 with
135     | .some v => P.map prod.fst <| pqs.v fst v
136     | .none => P.map (extend prod.1 prod.2 h) pqs.v snd
137     naturality m n' f := by
138     dsimp [N]
139     ext prod
140     change (n → m) ⊗ Fin m at prod
141     rcases prod with (ol, or)
142     dsimp [N]
143     -- We split on the case inPre (ol >> f) (f or)
144     split <=> rename_i heq
145     <=> simp only [CategoryStruct.comp] at heq
146     · simp only [inPre_comp, inPre_iff] at heq
147     subst heq
148     split <=> rename_i h
149     <=> simp only [inPre_defn, reduceCtorEq, Option.some.injEq] at h
150     subst h
151     simp
152     · rw [inPre_comp] at heq
153     split
154     <=> rename_i h'
155     · have := heq.symm.trans h'
156     contradiction
157     change _ = (P.map _ >> P.map _) _
158     rw [←P.map_comp]
159     apply congr (congr rfl _) rfl
160     change extend (ol >> f) _ _ = extend ol or _ >> f
161     apply Embedding.ext
162     intro i
163     induction i using Fin.cases
164     <=> simp [extend, CategoryStruct.comp]
165 }
166 invFun ntv := by
167     change NatTrans (coyonedat.obj (Opposite.op n) ⊗ N) P at ntv
168     change (Fin n → P.obj n) × P.obj (n + 1)
169     constructor
170     · exact fun i => ntv.app n (1 _, i)
171     · apply ntv.app (n + 1)
172     dsimp [N]
173     refine (Fin.succEmb _, 0)
174 left_inv psqv := by
175     dsimp
176     apply Prod.ext
177     · dsimp

```

```

178     ext i
179     split
180     <=> rename_i heq
181     <=> simp only [inPre_refl, reduceCtorEq, Option.some.injEq] at heq
182     subst heq
183     rw [P.map_id]
184     rfl
185     · dsimp
186     split
187     <=> rename_i heq
188     <=> simp at heq
189     conv =>
190       rhs
191       rw [←show _ = psqv.2 from funext_iff.mp (P.map_id _) psqv.2]
192       apply congr (congr rfl _) rfl
193       apply Embedding.ext
194       rintro (_|i)
195       <=> dsimp [extend]
196       <=> rfl
197   right_inv ntv := by
198     apply NatTrans.ext
199     funext m i
200     change (n → m) ⊗ Fin m at i
201     dsimp
202     split
203     <=> rename_i v heq
204     · change (ntv.app n >= P.map i.1) _ = _
205     rw [←ntv.naturality]
206     dsimp [N]
207     rw [Category.id_comp, inPre_iff.mp heq]
208     rfl
209     · change (ntv.app _ >= P.map _) _ = _
210     rw [←ntv.naturality]
211     dsimp [N]
212     rfl
213
214   noncomputable def PSq : Nat ⇒ Type where
215     obj := PSq0 P
216     map {X Y} f := objIso.invFun ∘ ((ihom N).obj P).map f ∘ objIso.toFun
217
218   variable {P}
219
220   noncomputable def iso : Iso (PSq P) (N →[_] P) where
221     hom := {
222       app m := objIso.toFun
223       naturality X Y f := by

```

```

224     change (objIso ◦ objIso.symm) ◦ ((ihom N).obj P).map f ◦ objIso
225     = ((ihom N).obj P).map f ◦ objIso
226     rw [Equiv.self_comp_symm]
227     rfl
228   }
229   inv := {
230     app n := objIso.invFun
231     naturality X Y f := by
232       change _ = objIso.symm ◦ _ ◦ objIso ◦ objIso.symm
233       rw [Equiv.self_comp_symm]
234       rfl
235   }
236
237   noncomputable def app : N ⊗ PSq P → P := (11 _ ⊗m iso.hom) >>
MonoidalClosed.uncurry (11 (N →[_] P))
238

```